



C++ Object-Oriented Programming (OOP) Practice

Welcome to my **C++ Object-Oriented Programming (OOP) Practice Repository**.

This repository contains my practice programs while learning **Object-Oriented Programming in C++**. Each program focuses on a specific OOP concept and demonstrates how it works using simple and understandable examples.



Topics Covered

1. Classes and Objects

A **class** is a blueprint used to create objects. It defines the data members (variables) and member functions (methods) that an object will have.

An **object** is an instance of a class that can access its data and functions.

Concepts

- Class Declaration
 - Object Creation
 - Member Variables
 - Member Functions
 - Access Specifiers
 - Constructors
 - Destructors
-

2. Encapsulation

Encapsulation is the process of combining data and functions into a single unit (class) while restricting direct access to the data.

Benefits

- Data Security
- Better Code Organization
- Controlled Access
- Easy Maintenance

Access Specifiers

- `private`
 - `protected`
 - `public`
-

3. Inheritance

Inheritance allows one class to acquire the properties and behaviors of another class.

Advantages

- Code Reusability
- Easy Maintenance
- Extensibility
- Hierarchical Design

Types of Inheritance

- Single Inheritance
 - Multiple Inheritance
 - Multilevel Inheritance
 - Hierarchical Inheritance
 - Hybrid Inheritance
-

4. Polymorphism

Polymorphism means "**Many Forms.**" It allows the same function or operator to behave differently depending on the context.

Compile-Time Polymorphism

- Function Overloading
- Operator Overloading

Run-Time Polymorphism

- Function Overriding
 - Virtual Functions
-

5. Operator Overloading

Operator overloading allows existing operators to work with user-defined objects.

Examples

- +
- -
- *
- ==
- <
- <<
- >>

6. Runtime Polymorphism

Runtime polymorphism is achieved using:

- Inheritance
- Virtual Functions
- Base Class Pointer

It enables dynamic method dispatch where the function to execute is determined at runtime.

7. Abstraction

Abstraction hides implementation details and exposes only the essential functionality.

In C++, abstraction can be implemented using:

- Abstract Classes
- Pure Virtual Functions
- Interfaces

Benefits

- Reduced Complexity
 - Improved Security
 - Better Code Design
 - Easier Maintenance
-

Repository Structure

```
Cpp-00P-Practice/  
|
```

```
├── 00_class.cpp
├── 01_encapsulation.cpp
├── 03_Inheritance.cpp
├── 04_types_inherit.cpp
├── 05_polymorphism.cpp
├── 06_operator_overloading.cpp
├── 07_runtime_polymorphism.cpp
├── 08_abstraction.cpp
├──
├── abstraction design/
│   ├── bird.h
│   ├── bird.cpp
│   └── main.cpp
├──
└── README.md
```

Technologies Used

- Language: **C++**
- Compiler: **g++ (MinGW)**
- IDE: **Visual Studio Code**
- Version Control: **Git**
- Repository Hosting: **GitHub**

Learning Objectives

This repository helped me understand:

- Object-Oriented Programming Fundamentals
 - Class Design
 - Encapsulation
 - Inheritance
 - Polymorphism
 - Operator Overloading
 - Runtime Polymorphism
 - Abstraction
 - Header Files
 - Code Organization
 - Reusable Programming Techniques
-

How to Compile and Run

Compile using g++:

```
g++ filename.cpp -o output
```

Run:

```
./output
```

On Windows PowerShell:

```
g++ filename.cpp -o output.exe  
.\output.exe
```



Why This Repository?

This repository serves as:

- My personal C++ learning journey
- A reference for OOP concepts
- Practice programs for interviews
- Revision notes
- Preparation for Data Structures and Algorithms (DSA)



Future Improvements

I plan to add:

- C++ Standard Template Library (STL)
- Exception Handling
- File Handling
- Templates
- Smart Pointers
- Lambda Expressions
- Design Patterns
- Data Structures
- Algorithms

- Mini Projects
-

Contributions

This repository is primarily for learning and practice. Suggestions, improvements, and constructive feedback are always welcome.

License

This project is available for educational purposes. Feel free to explore the code and use it for learning.

Author

Hariom Tiwari

Learning C++, Object-Oriented Programming, Data Structures & Algorithms, and continuously improving programming skills through hands-on practice.

★ If you found this repository useful, consider giving it a star!